

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\`a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_e2a9aaa7-6b2\agent-afffe4fbffbe20dd9.jsonl`  
- SHA-256: `317c0678623c314d71da05d4e20bf9e9c1163e8136edfb1330b0b2d97cffe058`  
- Source modified: `2026-05-30T19:13:19+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_e2a9aaa7-6b2`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-30T19:11:53.190Z)

Source Extractor

Research question: "Find promising AND reliable open-source tools/libraries for parsing Indian bank statement PDFs ? especially HDFC Bank savings/transaction statements ? into structured transaction rows (date, narration, ref, value date, withdrawal, deposit, closing balance), for reuse in a LOCAL-FIRST Python personal-finance tool ("muneem").

HARD CONSTRAINTS (a tool that violates these is unusable for us):

1. Must run 100% LOCALLY. Bank statements are highly sensitive ? NO cloud APIs, NO paid SaaS (e.g. bankstatementconverter.com, docparser, LLM extraction APIs). Paid/cloud is not acceptable even as a default.
2. Must be usable from Python.
3. License MUST be permissive & commercially safe: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL or that depends on copyleft native components ? in particular Camelot/tabula's reliance on Ghostscript (AGPL) or a Java runtime. PyMuPDF (AGPL) is already banned for us.

Specifically evaluate and compare these table-extraction approaches, for each giving: license (AND license of native deps), maintenance status (latest release + activity as of 2025/2026), reliability on SEMI-RULED / partially-borderless tables (HDFC's transaction table is not fully line-ruled), and password-protected-PDF support:

- pdfplumber (we already use it; MIT)
- Camelot ? camelot-py vs the maintained "pypdf_table_extraction" fork; its Ghostscript/OpenCV deps
- tabula-py (tabula-java + JVM)
- pypdfium2-based table extraction
- any other notable local options

Also assess the user's idea: "convert the PDF table to Excel/CSV first, then parse" ? is an intermediate CSV/Excel extraction more reliable than direct coordinate/text parsing, and which LOCAL+permissive tools do PDF?CSV/Excel table conversion well?

Finally, search for any existing, maintained, permissively-licensed open-source projects specifically for parsing Indian bank statements (HDFC/ICICI/SBI/Axis) or bank statements generally, that we could reuse or learn from (GitHub/PyPI).

Deliver a ranked shortlist of the most promising + reliable + license-safe options for HDFC statement parsing in a local Python tool, each with its license (and native-dep licenses), maintenance status, strengths/weaknesses for this use case, and a concrete recommendation for muneem (reuse library X vs. hand-roll coordinate clustering with pdfplumber). Cite sources."

Fetch and extract key claims from this source:

```
**URL:** https://unstract.com/blog/extract-tables-from-pdf-python/  
**Title:** How to Extract Tables from PDF Using Python ? A 2026 Guide
```

****Found via:**** license/dependency risk ? copyleft native deps search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-30T19:11:55.790Z)

I'll fetch the source and extract key claims relevant to the research question.

3. user (2026-05-30T19:12:07.708Z)

I appreciate the detailed request, but I notice the web page content you've provided is incomplete?it shows only the title "How to Extract Tables from PDF Using Python | A 2026 Guide" with a "[Content truncated due to length...]" placeholder.

****To provide accurate answers to your specific questions, I would need:****

1. The full article text covering each tool's discussion
2. Explicit statements about licenses, maintenance status, and release dates
3. Specific claims about table-type reliability (borderless vs. ruled)
4. Any mentions of password-protected PDF support
5. References to CSV/Excel conversion, Indian bank statements, or vendor recommendations
6. The actual publish date

****What I cannot do without the complete content:****

- Extract claims about specific tools (pdfplumber, Camelot, tabula-py, etc.)
- Verify license information for dependencies like Ghostscript or Java
- Determine if this is vendor marketing (Unstrat blog) versus neutral guidance
- Identify whether paid products are recommended over open-source options
- Provide properly-attributed quotes under your 125-character limit

****Could you please provide the complete article text?**** That way I can systematically extract the specific information you're seeking while respecting both your formatting constraints and the source material.

4. assistant (2026-05-30T19:12:10.349Z)

The content was truncated. Let me try a more targeted fetch focusing on specific tools to get the actual content.

5. user (2026-05-30T19:12:14.359Z)

I appreciate your request, but I need to be transparent: the web page content you've provided appears to be truncated ("Content truncated due to length..."), so I don't have access to the actual article text about Camelot, tabula-py, pdfplumber, their dependencies, or their performance characteristics with different table types.

To provide accurate information with proper quotes and citations, I would need the complete article content. Without it, I cannot:

- Quote exact sentences about each library's strengths/weaknesses
- Confirm what the article states about Ghostscript and Java dependencies

- Verify specific claims about ruled vs. borderless table handling

If you could share the full section text, I'd be happy to summarize it according to your specifications, including the 125-character quote limit and proper attribution.

6. assistant (2026-05-30T19:12:17.379Z)

The WebFetch tool is truncating the content before the model can read it. Let me try fetching the raw content directly via the Bash tool to bypass the truncation, since this is a non-sensitive public blog page.

7. user (2026-05-30T19:12:20.416Z)

2400211 /tmp/unstract_tables.html

8. assistant (2026-05-30T19:12:22.942Z)

Downloaded 2.4MB of HTML. Let me extract the readable text content, focusing on the relevant sections about the libraries.

9. user (2026-05-30T19:12:28.065Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 3, in <module>
    with open('/tmp/unstract_tables.html', encoding='utf-8') as f:
        ~~~~^~~~~~
FileNotFoundError: [Errno 2] No such file or directory: '/tmp/unstract_tables.html'
```

10. assistant (2026-05-30T19:12:30.600Z)

The Bash tool resets working directory between calls and `/tmp` may not be consistent on Windows. Let me use an absolute path and chain the commands.

11. user (2026-05-30T19:12:36.371Z)

```
LEN 2399256
datePublished 2024-07-22T13:54:32+00:00
dateModified 2026-04-30T11:42:15+00:00
meta-date 2024-07-22T13:54:32+00:00
meta-date 2026-04-30T11:42:15+00:00
```

12. assistant (2026-05-30T19:12:39.008Z)

Date confirmed: published 2024-07-22, modified 2026-04-30 (consistent with a "2026 Guide"). Now let me extract the article body text to find claims about each library.

13. user (2026-05-30T19:12:42.003Z)

```
TEXTLEN 37956
549 unstract_text.txt
```

14. assistant (2026-05-30T19:12:44.203Z)

Now let me search the extracted text for the key libraries and topics.

15. user (2026-05-30T19:12:46.552Z)

130: That's why Python PDF table extraction is rarely as simple as it seems. The good news: the Python ecosystem has several libraries designed specifically for this job. Camelot, Tabula, and Pdfplumber each approach the problem differently ? some use computer vision, others rely on coordinate analysis or layout heuristics. But no single tool handles every PDF.

134: Camelot

135: Tabula

143: When it comes to getting tables out of PDF files, Python has several strong libraries that can make the process easier. Each of these libraries has its own set of features, advantages, and limitations. In this section, we'll introduce you to four well-known Python parsers for getting tables out of PDFs: Camelot, Tabula, Pdfplumber, and Pdftables.

144: Camelot

145: Camelot is a Python library made specifically for getting tables out of PDF files. It works great with simple table structures and has a user-friendly interface. Camelot's main features include:

149: Tabula

150: Tabula is another popular library for getting tables out of PDFs, known for being easy to use and strong. It uses Java but provides a convenient Python wrapper. Tabula's main features include:

175: Camelot: PDF Table Extraction for Humans ¶

176: Camelot is a Python library created for getting tables out of PDF files. It's known for being simple and effective, especially when working with PDFs that have basic table structures. Camelot uses computer vision algorithms to find and get tables from PDF documents, making it a reliable choice for many table extraction tasks.

177: Key features of Camelot include:

178: Table Parsing: Camelot can accurately get tables from PDF files, even those with complex structures.

180: Visual Debugging: Camelot lets you see and adjust the table extraction process.

182: Camelot Installation Steps

183: Before you can install Camelot, you need to make sure that the dependencies are installed, so make sure to have Ghostscript installed.

184: Then you can install Camelot using the following command:

185: `pip install camelot-py[cv]`

189: It is because there is currently an incompatibility of Camelot with PyPDF2 2.3.0.0, so you might need to specify an older version of PyPDF2:

192: Here's a simple example of how to use Camelot to get tables from a PDF file:

193: `import camelot`

195: `tables = camelot.read_pdf('best-unicef-1.pdf')`

201: `import camelot`: This line imports the Camelot library.

202: `tables = camelot.read_pdf('best-unicef-1.pdf')`: The `read_pdf` function from the Camelot library is used to read the PDF file `best-unicef-1.pdf` and extract tables from it. The extracted tables are stored in the `tables` variable as a list of table objects.

205: Camelot's ease of use and strong features make it an excellent choice for getting tables from PDFs, especially when dealing with simple to moderately complex table structures. In the next section, we will explore Tabula, another popular library for getting tables out of PDFs.

206: Tabula

207: Tabula is a popular tool for getting tables out of PDF files. It started as a web application but has a strong Python wrapper called `tabula-py`, which lets you use its features in Python. Tabula is known for handling multi-page PDFs well and having a user-friendly approach to table extraction.

208: Key features of Tabula include:

209: Java-Based Backend: Tabula uses a Java library for processing PDFs, which can handle many types of table structures.

212: Ease of Use: The Python wrapper `tabula-py` provides a simple API for using Tabula's capabilities in Python projects.

213: Tabula Installation Steps

214: To use Tabula in Python, you need to install the `tabula-py` package. Since Tabula relies on Java, you also need to have Java Runtime Environment (JRE) installed on your system. You can install `tabula-py` using the following command:

215: `pip install tabula-py`

216: Make sure that Java is properly installed and set up in your system's PATH.

218: Here's a simple example of how to use Tabula to get tables from a PDF file:

219: `import tabula`

221: `tables = tabula.read_pdf('best-unicef-1.pdf', pages='all')`

227: `import tabula`: This line imports the `tabula` library.

228: `tables = tabula.read_pdf('best-unicef-1.pdf', pages='all')`: The `read_pdf` function from the `tabula` library is used to read the PDF file `best-unicef-1.pdf` and extract tables from it. The `pages='all'` parameter ensures that tables from all pages of the PDF are extracted. The extracted tables are stored in the `tables` variable as a list, with each item representing a table.

231: Tabula's combination of ease of use and strong functionality makes it a popular choice for getting tables from PDFs, especially for documents with multiple pages. In the next section, we

will explore Pdfplumber, another powerful library for getting tables out of PDFs.

```

241: pip install pdfplumber
245: import pdfplumber
247: with pdfplumber.open('best-unicef-1.pdf') as pdf:
255: import pdfplumber : This line imports the pdfplumber library.
256: with pdfplumber.open('best-unicef-1.pdf') as pdf: : This line opens the PDF file ?best-unicef-1.pdf? using the open function from the pdfplumber library. The opened file is assigned to the variable pdf.
317: To see how well the four Python libraries (Camelot, Tabula, Pdfplumber, and Pdftables) and LLMWhisperer work, we'll use two sample documents:
340: Camelot
346: Results from PDF table extraction using Camelot
347: Tabula
348: Ease of Use: It's very user-friendly. The installation is simple, and it has a straightforward API. However, it requires Java, which can be an additional step for some users.
352: Results from PDF table extraction using Tabula
366: Library Ease of use Accuracy Output Structure Notes LLMWhisperer Very high Very high Very high Excellent for complex tables Built-in support for OCR Can directly provide output as input to an LLM Camelot High Medium Medium Good for simpler tables Tabula High Medium Medium Some formatting issues with complex tables Pdfplumber Medium Very High Very High Excellent for complex tables; detailed control Pdftables Very High High High Requires API key; very user-friendly
367:[Omitted long matching line]
372: Camelot
373: Documentation : Camelot Documentation
374: GitHub Repository : Camelot GitHub
375: Tabula
376: Documentation : Tabula Documentation
377: GitHub Repository : Tabula GitHub
401: The blog tested all libraries against Apple's financial statements. Camelot failed entirely on that document. Tabula extracted data but introduced formatting issues. Pdfplumber delivered very high accuracy. LLMWhisperer scored extremely well across both test documents. So if you regularly work with dense financial PDFs, Pdfplumber or LLMWhisperer are your safest bets for reliable python pdf table extraction.
402: Can I extract data from borderless or irregular tables using python pdf table extraction tools without manual tuning?
403: Camelot offers a ?stream? mode for borderless tables, but it often needs parameter adjustments. Tabula generally struggles with irregular layouts. Pdfplumber gives you fine-grained control ? you can manually define table regions if needed. If you want to avoid configuration entirely and just learn how to read tables from pdf using python without the hassle, LLMWhisperer preserves layout automatically regardless of borders or weird structures, making it the most hands-off option.
405: Tabula requires a working Java installation. Camelot needs Ghostscript and OpenCV. Pdfplumber has moderate dependencies (pillow, pdf2image). LLMWhisperer needs only pip install llmwhisperer-client plus an API key ? no Java, no Ghostscript, no system-level headaches. If you want to minimize setup time while still learning how to extract tables from pdf using python, that matters.
407: Avoid per-document overhead. Batch your PDFs and use multiprocessing with Camelot or Pdfplumber. Tabula is slower at scale because each call spins up a Java JVM. If you don't want to manage infrastructure, LLMWhisperer's API scales automatically ? just send files and get back JSON, no server maintenance required. For teams that need to extract tables from pdf python at high volume, the API route saves significant engineering time.
409: Traditional libraries extract only text. Pdfplumber ignores images entirely. Camelot does the same. For mixed content, you need an OCR-aware approach. LLMWhisperer processes the full page layout, including any text embedded inside images, and preserves the table structure in the output. So if you're wondering how to read tables from pdf using python for scanned or image-heavy documents, LLMWhisperer is the only tool in this comparison built for that scenario.

```

16. assistant (2026-05-30T19:12:50.569Z)

I have a comprehensive view now. This is clearly the Unstract vendor blog promoting their LLMWhisperer product alongside the open-source libraries. Let me capture a couple more specific details ? the comparison table and any pdfplumber description and license mentions ? to confirm there's no explicit license discussion.

17. user (2026-05-30T19:12:52.234Z)

232 Pdfplumber

233 Pdfplumber is a strong and flexible Python library designed to get text and tables from PDF files. It's known for its accuracy and ability to handle complex table structures that other libraries may have trouble with. Pdfplumber gives you detailed control over the extraction process, making it great for users who need to work with complicated or non-standard table layouts.

234 Key features of Pdfplumber include:

235 High Precision: Can accurately extract complex table structures.

236 Detailed Control: Gives you fine-grained control over the extraction process, allowing you to handle specific table layouts.

237 Versatility: Supports getting both text and tables from PDFs.

238 Output Options: The data you get can be saved in various formats, including CSV and JSON.

239 Installation Steps

240 To install Pdfplumber, you can use pip, Python's package installer. Run the following command in your terminal:

241 `pip install pdfplumber`

242 Pdfplumber also depends on pillow and pdf2image, which will be installed automatically with the above command.

243 Code Example for Extracting Tables from a PDF

244 Here's an example of how to use Pdfplumber to get tables from a PDF file:

245 `import pdfplumber`

246 `# Open the PDF file`

247 `with pdfplumber.open('best-unicef-1.pdf') as pdf:`

248 `# Go through each page`

249 `for page in pdf.pages:`

250 `# Get tables from the current page`

251 `tables = page.extract_table()`

252 `# Print the table data`

253 `print(tables)`

254 Here's a step-by-step description of what the code does:

255 `import pdfplumber` : This line imports the pdfplumber library.

256 `with pdfplumber.open('best-unicef-1.pdf') as pdf:` : This line opens the PDF file 'best-unicef-1.pdf' using the open function from the pdfplumber library. The opened file is assigned to the variable pdf.

257 The for loop iterates through each page in the PDF file:

258 `page.extract_table()` : This method extracts tables from the current page.

259 `print(tables)` : This line prints the data of the tables.

340 Camelot

341 Ease of Use: It's easy to install and use. The documentation is clear.

342 Accuracy: It has high accuracy for simpler tables but struggles with more complex structures. You might need to make manual adjustments for the best results.

343 Could not read tables from file apple.pdf

344 Output Structure: It preserves the structure and formatting of simpler tables well. However, complex tables may require some post-processing to clean up the data.

345 Screenshot from reading file best-unicef-1.pdf :

346 Results from PDF table extraction using Camelot

347 Tabula

348 Ease of Use: It's very user-friendly. The installation is simple, and it has a straightforward API. However, it requires Java, which can be an additional step for some users.

349 Accuracy: It has good accuracy for simple to moderately complex tables. However, it struggles with highly complex or irregular tables.

350 Output Structure: It maintains the table structure reasonably well for simpler tables. Some formatting issues can occur with more complex tables.

351 Screenshot from reading file best-unicef-1.pdf :

352 Results from PDF table extraction using Tabula

353 Pdfplumber

354 Ease of Use: The installation is straightforward. The API provides detailed control, but you might need some familiarity with PDF structures to get the best results.

355 Accuracy: It has very high accuracy, especially with complex and mixed data tables. It consistently extracts both simple and complex tables accurately.

356 Output Structure: It maintains the original table layout and structure precisely. The outputs are clean and well-organized.

357 Screenshot from reading file best-unicef-1.pdf :

358 Results from PDF table extraction using Pdfplumber
359 Pdftables
360 Ease of Use: It's extremely easy to use. It has a simple API, but you need an API key. The process to get the key is quick and seamless. It offers 40 free pages.
361 Accuracy: It has high accuracy across different types of tables. It handles mixed data types and complex tables effectively.
362 Output Structure: It preserves the original table structure effectively. The outputs are clean and organized, with minimal need for additional formatting.
363 Screenshot from reading file best-unicef-1.pdf :
364 Results from PDF table extraction using pdftables
365 Extract Tables from PDF in Python: Library Comparison Table
366 Library Ease of use Accuracy Output Structure Notes LLMWhisperer Very high Very high Very high Excellent for complex tables Built-in support for OCR Can directly provide output as input to an LLM Camelot High Medium Medium Good for simpler tables Tabula High Medium Medium Some formatting issues with complex tables Pdfplumber Medium Very High Very High Excellent for complex tables; detailed control Pdftables Very High High High Requires API key; very user-friendly
367 LLMWhisperer performed the best in terms of accuracy and output structure. It's also extremely user-friendly with a simple API and a generous free tier. Pdfplumber also performed exceptionally well, particularly with complex tables, offering detailed control over the extraction process. Camelot and Tabula are easier to use but may require adjustments for more complex tables, while Pdftables provides a highly accurate and user-friendly solution, ideal for users seeking quick and reliable results with minimal setup. Each library has its strengths, and the choice depends on the specific requirements and complexity of your PDF table extraction tasks. For high accuracy and flexibility, LLMWhisperer and Pdfplumber are excellent choices, while Camelot, Tabula, and Pdftables offer strong performance for simpler extraction needs.
368 Live coding session on PDF Table extraction
369 You can also watch this live coding webinar where we explore all the challenges involved in PDF parsing and compare various PDF parsing tool's capabilities.

18. assistant (2026-05-30T19:12:55.126Z)

I now have everything needed. Let me clean up the temp files and compile the findings.

19. user (2026-05-30T19:12:55.985Z)

cleaned

20. assistant (2026-05-30T19:13:03.729Z)

I have thoroughly analyzed the source. Here is my assessment.

****Source quality: blog (vendor marketing).**** This is the Unstract company blog. Unstract sells "LLMWhisperer," a cloud/API PDF-extraction product that the article repeatedly inserts into an otherwise generic open-source library roundup, consistently ranking it "Very high" on every axis and steering readers toward the paid API ("just send files and get back JSON, no server maintenance required... the API route saves significant engineering time"). The open-source comparisons are real and roughly accurate, but the framing is promotional and the test methodology is shallow (two sample PDFs, no version numbers, no license discussion). Notably, the article never discusses licenses at all ? a key gap for the research question. Published 2024-07-22, modified 2026-04-30.

The claims relevant to muneem are the open-source dependency and reliability statements, which corroborate (from an independent vendor source) the copyleft/JVM dependency risks and pdfplumber's strength on complex tables.

21. user (2026-05-30T19:13:11.799Z)

Structured output provided successfully

22. assistant (2026-05-30T19:13:19.793Z)

I've completed the source extraction. Summary of what I found:

****Source:**** Unstract blog, "How to Extract Tables from PDF Using Python | A 2026 Guide" ? published 2024-07-22, last modified 2026-04-30.

****Quality rating:** blog (vendor marketing).****** Unstract sells LLMWhisperer, a cloud/API extraction product that the article inserts into a generic open-source roundup and ranks "Very high" on every axis while steering readers to the paid API. The open-source comparisons are independently useful but the framing is promotional, the test corpus is shallow (two PDFs, no versions), and critically the article never discusses licenses ? a gap for muneem's research question.

****Five falsifiable claims extracted**** (3 central, 2 supporting), all bearing on the research question:

- Dependency confirmation: Camelot needs Ghostscript + OpenCV; tabula-py needs Java (central ? corroborates the copyleft/JVM risk muneem must avoid).
- Borderless-table handling: Camelot stream mode needs tuning, Tabula struggles, pdfplumber allows manual region definition (central ? supports hand-rolling with pdfplumber).
- Financial-PDF benchmark: Camelot failed entirely on Apple's statements, Tabula had formatting errors, pdfplumber was very high accuracy (central ? independent evidence pdfplumber is the safest local library for financial tables).
- pdfplumber exports to CSV/JSON (supporting ? addresses the "convert to CSV first" idea).
- Tabula is slow at scale due to per-call JVM spin-up (supporting).

Findings delivered via StructuredOutput.